

电子科技大学

实验报告

指导教师：邓健

一、实验项目名称：ALU 设计

二、实验目的：

掌握采用 Verilog 进行 ALU 设计和仿真的方法。

三、实验内容：

分析 MIPS 指令的逻辑功能，定义 ALU 的功能码，使用 Verilog 和 ISE 开发工具进行逻辑设计和仿真。

四、实验原理：

ALU 是负责运算的电路。根据 MIPS 指令的功能，ALU 至少需实现以下的运算：ADD（加）、SUB（减）、AND（与）、OR（或）、XOR（异或）、LUI（置高位立即数）、SLL（逻辑左移）、SRL（逻辑右移）和 SRA（算术右移）。

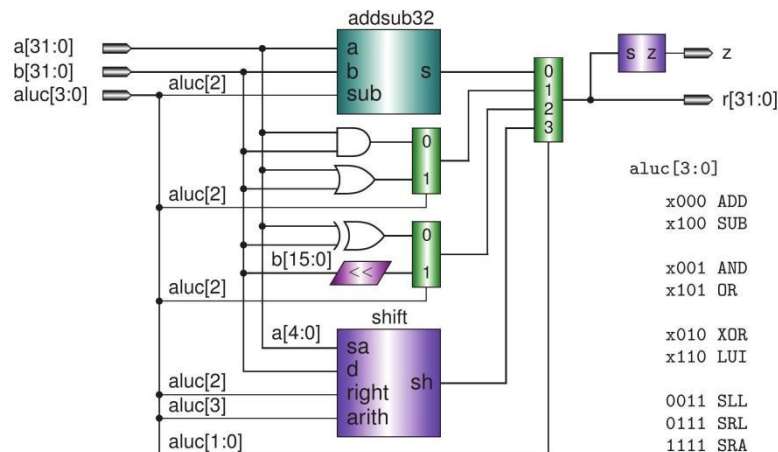


图 1 ALU 的逻辑电路图

$a[31:0]$ 和 $b[31:0]$ 是两个 32 位的输入， $aluc[3:0]$ 是 ALU 的操作控制码， $r[31:0]$ 是 ALU 的 32 位输出， z 是一位零标志。当 $r[31:0]$ 为 0 时， z 输出 1。

ALU 设计的基本想法是准备好所有的运算电路，然后用多路器进行选择。多

路器的选择信号就是 ALU 的操作控制码。

五、实验器材（设备、元器件）：

PC 机、Windows XP、Anvy1 开发板、Xilinx ISE 14.7 开发工具、Digilent Adept 下载工具。

六、实验步骤：

实验步骤包括：建立新工程、设计代码与输入、约束与实现、生成流代码与下载调试。

首先，建立一个新项目

根据给出的代码，补全相关的模块即可。

对 `aluc` 经过分析可知，加与减法的 `aluc` 的后两位是 `00`，同理与和或操作后两位是 `01`，置高位和异或是 `10`，剩余的三个移位操作是 `11`。

因此可以设置一个选择器根据 `aluc` 的后两位减小范围。

对于 `addsub32` 模块，实现的是加减法，两个操作数也就是两个输入 `a`，`b`。之后是根据输入的 `aluc` 判断二者的第三位不一致，所以输入一个 `aluc[2]` 来判断是加法还是减法还有一个输出。具体的加减法代码在后面源代码里就是根据 `sub` 是 `1` 则为减法，为 `0` 则为加法。

七、关键源代码：

以下是 ALU 的 Verilog HDL 代码

```
module alu (a,b,aluc,r, z) ;
    input [31 : 0] a , b ;
    input [3:0] aluc ;
    output [31: 0] r;
    output z;

    wire [31 : 0] d_and = a & b ;
    wire [31 : 0] d_or = a | b;
    wire [31 : 0] d_xor = a ^ b;
    wire [31 : 0] d_lui = {b[15 : 0] ,16'h0};
    wire [31 : 0] d_and_or = aluc[2]? d_or : d_and ;
    wire [31 : 0] d_xor_lui = aluc[2]? d_lui : d_xor ;

    wire [31 : 0] d_as,d_sh;

    addsub32 as32 (a,b,aluc[2] , d_as) ;
    shift shifter (b ,a [4 : 0] ,aluc[2] ,aluc [3] ,d_sh );
    mux4x32 select ( d_as ,d_and_or, d_xor_lui ,d_sh , aluc[1:0] , r);

    assign z = ~|r;

endmodule

module addsub32(a,b,sub,s);

    input[31:0] a,b;
    input sub;
    output reg[31:0] s;

    always @* begin
        if (sub)
            begin
                s=a-b;
            end
        else
            begin
                s=a+b;
            end
    end

end

module shift(d , sa, right,arith,sh);
    input [ 31:0] d;
    input [ 4:0 ] sa ;
    input right, arith ;
    output [31 : 0] sh ;
    reg [31 : 0] sh ;
    always @* begin
        if ( !right ) begin
            sh = d << sa ;
        end else if ( !arith ) begin
            sh = d >> sa ;
        end else begin
            sh = $signed(d) >>> sa ;
        end;
    end

end

endmodule

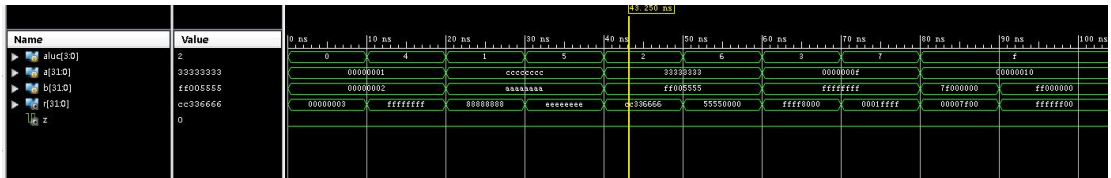
module mux4x32(a0, a1, a2 , a3 , s, y);
    input [31:0] a0 ,a1 , a2 , a3 ;
    input [1:0] s ;
    output [31:0] y;

    function [31:0] select ;
        input [31:0] a0, a1 , a2 , a3 ;
        input [1:0] s ;
        case ( s )
            2'b00 : select = a0;
            2'b01 : select = a1 ;
            2'b10 : select = a2 ;
            2'b11 : select = a3 ;
        endcase
    endfunction

endfunction
```

八、实验结论：

仿真结果如下：



九、总结及心得体会：

对于 alu 的实现更加的了解，充分理解了其原理。

十、对本实验过程及方法、手段的改进建议：

无

报告评分：

指导教师签字：